UNIVERSITÉ DE NANTES
12, rue de la Houssinière
44322 Nantes

LABORATOIRE D'INFORMATIQUE
DE NANTES ATLANTIQUE
UFR de Sciences et Techniques
2, rue de la Houssinière
44322 Nantes

Initiation à la recherche

Rapport

Florian Barbin
Damien Le Quéré
Victor Picault
Myrtille Ponge
2008-2009

Master 1 - ALMA

Table des matières

1 Introduction **3**

2 Environnement de réalisation du stage **5**
2.1 Le module d'introduction à la recherche 5
2.2 Notre équipe de recherche 5
2.3 Présentation du TALN 6

3 État de l'art **7**
3.1 L'article de Giguet et Lucas 7
3.1.1 Synthèse de l'article 7
3.1.2 Caractéristique de (Giguet et Lucas, 2004) 8
3.1.3 Ce qui nous intéresse dans l'article 8
3.2 L'article de N. Hernandez et F. Poulard 8
3.2.1 Synthèse de l'article 8
3.2.2 Les principales caractéristiques de (Poulard et Hernandez, 2008) 9
3.2.3 Comparaison avec (Giguet et Lucas, 2004) 9
3.2.4 Ce qui nous intéresse dans l'article 9
3.3 L'article de R. Mitkov 9
3.3.1 Synthèse de l'article 9
3.3.2 Ce qui nous intéresse dans l'article 10
3.4 Le mémoire de F. Poulard 10
3.4.1 Contenu 10

4 Observation d'un corpus de textes **12**
4.1 Objectifs de l'observation 12
4.2 Résultats 12
4.3 À retenir 13

5 Approches **15**
5.1 Précision du sujet 15
5.2 Sous-problèmes et nos choix 15
5.2.1 La détection de citation 15
5.2.2 La résolution d'anaphores pronominales 16
5.3 Implémentation 16
5.3.1 Choix techniques 16
5.3.2 Notre travail 18

6 Évaluation **24**
6.1 Détection du discours repris 24
6.2 Résolution des anaphores pronominales 26

7 Analyse des résultats **27**
7.1 Détection du discours repris 27
7.2 Résolution des anaphores pronominales 28

8 Conclusion **30**
8.1 Sur notre travail 30
8.2 Nos observations du travail de recherche 30

Chapitre 1

Introduction

Notre problème est le suivant :

"La résolution d'anaphores pronominales
dans le cadre de la détection de citations"

Nous cherchons à savoir qui dit quoi dans un texte lorsque la personne qui reprend le discours n'est pas précisée directement mais est remplacée par un pronom (c'est ce que nous appelerons une anaphore pronominale), exemple :

> **Jacques Chirac** a rencontré Jean Pierre Raffarin, il lui a assuré qu'il allait faire beau.
> "Je ne pourrais supporté qu'il en soit autrement" a-t-**il** ajouté.

Dans cet exemple nous devons repérer que le dernier 'il' (que nous nommerons *expression locuteur pronominale* par la suite) se réfère à Jacques Chirac (que nous appelerons *entité nommée de référence* ou simplement *référence*). La partie entre guillemets sera appelée *discours repris*. Nous Considèrerons une *citation* comme une portion de texte constituée d'une expression locuteur, d'un discours repris et d'un segment citationnel les reliant (approche de Fabien Poulard et Nicolas Hernandez dans "Qui dit quoi ?" 2008)

Notre problème revient donc à **trouver l'entité nommée correspondant à l'expression locuteur d'une citation lorsque cette expression est un pronom**.

Pour résoudre ce problème nous devons être capable :
- de détecter les citations (le discours repris et l'expression locuteur)
- de résoudre des anaphores pronominales

C'est donc sur ce problème que nous nous sommes penché durant 4 mois, dans ce rapport nous présenterons d'abord les conditions dans lesquelles nous avons réalisé le stage, puis nous présenterons le travail bibliograhique que nous avons effectué et les intérêts de chaque lecture dans la résolution de notre problème.

Ensuite nous verrons le bilan de l'observation d'un corpus de textes tout en précisant l'intérêt de cette observation, puis nous présenterons notre travail en lui même : les différents choix que nous avons fait, les approches dont nous nous sommes inspiré pour les différents sous-problèmes et l'implémentation de ce travail.

Enfin nous evaluerons ce travail pour chacun de ses sous-problèmes et nous analyserons ces résultats, puis nous conclurons sur différentes observations du travail d'un chercheur.

Chapitre 2

Environnement de réalisation du stage

2.1 Le module d'introduction à la recherche

La formation dans laquelle nous sommes actuellement à savoir le master Architectures Logicielles (ALMA) est une formation à vocation à la fois recherche et professionnel. En effet, celui-ci n'a pas réellement de spécialité dans le sens ou il n'a pas d' *étiquette* qu'elle soit professionnel ou recherche. Cette formation permet en réalité aux étudiants ayant des perspectives de carrière différentes de leur offrir à la fois une immersion professionnel en seconde année et un aperçu des métiers de la recherche au travers d'un module d'Initiation à la recherche (IR) en première année.

Le module d'IR a pour but de présenter aux étudiants les métiers de la recherche. En effet, dans un premier temps nous nous sommes vu dispenser 8 heures des cours nous présentant les métiers de la recherche, le fonctionnement de la recherche publique et privée en France et à l'étranger ainsi que les thématiques de recherche. Dans un second temps nous avons intégré une équipe de recherche pour une période de 4 mois à raison d'une journée et demi par semaine.

2.2 Notre équipe de recherche

Pour ce module d'introduction à la recherche nous avons été rattachés à l'équipe du Laboratoire Informatique de Nantes Atlantique (LINA) spécialisée dans le Traitement Automatique du Langage Naturel (TALN). Cette équipe dirigée par Béatrice DAILLE, s'oriente sur différents axes de recherche qui sont :

- Théorie des langages caractérisation des modèles linguistiques du point de vue de leur expressivité et des liens qu'ils entretiennent avec d'autres formalismes.
- Analyse et acquisition lexicale méthodes d'alignement et de détection dans les textes d'unités lexicales non présentes dans les dictionnaires comme celles relevant de domaines de spécialités ou de nouvelles formes d'expressions langagières.
- Analyse et acquisition grammaticale méthodes d'inférences robustes de grammaires lexicalisées avec une adaptation de modèles stochastiques ou mixte à diverses applications.
- Analyse du discours et des documents définition de modèles prédicatifs du discours, méthodes d'analyse sémantique-textuelle pour la détection des opinions ou la reprise de contenu.

Nous avons plus particulièrement été rattaché à la branche analyse du discours et des documents sous la responsabilité de Monsieur Hernandez Nicolas, maitre de conférence à l'université de Nantes ainsi que Monsieur Poulard Fabien, doctorant en TALN.

2.3 Présentation du TALN

Le TALN est une discipline à la frontière de plusieurs domaines distincts les uns des autres mais ayant une problématique commune qui est le traitement de la langue humaine. Ainsi, la linguistique, l'informatique et l'intelligence artificielle se rejoignent par l'application des techniques de chaque domaine afin de répondre d'une façon commune et automatisé aux problèmes liés au langage humain. Les applications les plus typique du TAL sont :

- la traduction automatique
- la correction orthographique ;
- la recherche d'information et la fouille de textes
- la reconnaissance vocale ;
- la reconnaissance de l'écriture manuscrite

Chapitre 3

État de l'art

Nous avons commencé notre travail de recherche par la lecture des trois articles suivants :
- « La détection de citation et des locuteurs dans les textes informatifs »de E. Giguet et N. Lucas (2004),
- « Qui dit quoi »de N. Hernandez et F. Poulard(2008),
- « Robust pronoun resolution with limited knowledge »de Rusian Mitkov,
- « Repérage automatique de citations dans des documents journalistiques »de Fabien Poulard (2007).

Les deux premiers articles s'intéressent à la détection de discours repris tandis que le troisième présente la détection d'anaphore. La lecture de ces articles nous a permis de comprendre le sujet, d'apprendre le vocabulaire spécifiques et les problèmes spécifiques au traitement du langage. Contrairement à ce qu'on est habitué à faire, on ne peut pas avoir de solution exacte.

3.1 L'article de Giguet et Lucas

3.1.1 Synthèse de l'article

L'article que nous allons synthétisé est "La détection de citation et des locuteurs dans les textes informatifs" de E. Giguet et N. Lucas (2004).
Giguet et Lucas présente une approche syntaxique après avoir constaté les limites de l'approche classique. L'approche classique, appelée stratégie lexical est la plus utilisée, elle fait appel à des listes en mémoire pour identifier des élèments au sein d'une phrase. Des ressources lexicales permettent le ciblage de la source et du verbe et les verbes citatifs sont repertoriés. L'algorithme pour la détection de la source et du discours repris se déroule en deux étapes :
- Catégorisation des mots ou des groupes de mots présents dans le lexique,
- Identification des mots stockés à partir des mots catégorisés.

Cette approche est malheureusement peu robuste, peu maintenable et coûteuse car elle ne s'adapte pas à la grande variabilité des sources.

Giguet et Lucas présentent donc une nouvelle approche, plus souple, appelée stratégie syntaxique. Cette approche cherche à caractériser les formes très variables des citations et des locuteurs non pas par un lexique mais par la recherche de constante appelé aussi invariant. Ils cherchent à identifier la source, le discours rapporté et le relateur.
Cette stratégie utilise les critères phrasiques qui sont divisés en trois classes :
- Indices typographiques,

– Indices morphosyntaxiques,
– Indices positionnels.

On utilise la co-présence de ces trois indices et leurs positions relatives déclenchent l'affectation des variables sources, discours repris et relateur.

3.1.2 Caractéristique de (Giguet et Lucas, 2004)

Ils ont des règles basés sur des motifs.

Ils séparent la détection en deux phases :
– Repérage de Source Relateur Discour.
– Recherche des indices typographique, morphosyntaxiques et positionnels.

Ils recherchent des invariants et les mettent en relation.

Ils utilisent un dictionnaire pour la caractérisation et le repérage des candidats.

3.1.3 Ce qui nous intéresse dans l'article

Nous pensons utiliser les critères phrastique décrit ci-dessus pour la recherche et de caractérisation des candidats.

3.2 L'article de N. Hernandez et F. Poulard

3.2.1 Synthèse de l'article

L'article que nous allons synthétisé est "Qui dit quoi " de N. Hernandez et F. Poulard(2008). Alors que les travaux précédents ont surtout reposé sur des ressources lexicales, syntaxiques et sémantiquse (N. Hernandez et F. Poulard,2008) cherchent à exploiter les marques de surfaces de nature typographique, morphologique et positionnelle. Ils suivent pour cela l'approche décrite plus haut de (Giguet et Lucas, 2004).

Les travaux de (Giguet et Lucas, 2004) reposent sur des règles et co-présence de marque et proposent de représenter une citation comme une séquence de trois objet citationnel élémentaire : la source, le discours rapporté et le relateur. (Hernandez et Poulard, 2008) pensent que la cooccurence des composants suffit à reconstituer un segment approchant la forme canonique, le segment citationel. Ils proposent de conserver l'approche composant mais pas le motif Source + Discours repris + Relateur. D'après des observations sur leur corpus, (Hernandez et Poulard, 2008) mettent en évidence deux structures qui ressortent pour le discours direct et indirect :
– Les relateurs ponctuatifs ou verbaux mettent en relation l'expression locuteur et le discours repris,
– La juxtaposition de l'expression locuteur et du discours repris.

Les caratéristiques suivantes ressortent pour les expressions locuteurs :
– Les expressions locuteurs sont majoritairement des groupes nominaux et noms capitalisés.
– Parfois il n'y a pas de locuteur.

En considérant la diversité des formes et des marques de la citation, (Hernandez et Poulard, 2008) choisissent un apprentissage supervisé pour la détection du discours repris. Lors de la phase de prétraitement, les articles à analyser sont découpés en mots et en phrases puis étiquetés morphosyntaxiquement avec l'outil TreeTagger. Ces chaînes servent premièrement à caractériser les données pour construire des modèles puis elles servent à traiter de nouvelles données.

(Hernandez et Poulard, 2008) ont choisit de caractériser uniquement les textes entre guillemets utilisés comme candidats au discours repris.

Vocabulaire :
- **Locuteur** : Personne source de l'énonciation dans l'énoncé rapporté.
- Epression locuteur : expression linguistique introduisant un locuteur et faisant référence à l'énonciateur de la situation d'énonciation source.
- **Discours source** : Les éléments du discours source repris tel quel pour être intégré au nouveau discours.
- **Discours repris** : Résultat de l'intégration du discours sources dans le discours de la situation d'énonciation de référence.

3.2.2 Les principales caractéristiques de (Poulard et Hernandez, 2008)

Le relateur est introduit après le repérage du candidat.

Il ne considère pas le motif Source + Discours repris + relateur, il ne considère pas d'ordre dans les élèments de la citation.

Les entités considérés sont : expressions locuteur et discours repris.

Les expressions locuteurs sont candidates aux entités nommées.

Utilisation d'algorithme d'apprentissage et d'un corpus annoté.

La détection de citations et séparés en deux phases distinctes :
- A chaque indice, on se pose la question : on applique le modèle ? oui/non
- Puis on se pose la question : est-ce un discours repris ? oui/non

Ils utilisent des dictionnaires et des marques de surface pour la caractérisation.

3.2.3 Comparaison avec (Giguet et Lucas, 2004)

La première étape est identique, c'est-à-dire la recherche des candidats et leur caractérisation. Ensuite, ils ont deux approches différentes pour atteindre le même objectif :
- Symbolique pour Giguet et Lucas (application de règles pour retrouver des motifs).
- Statistique avec des algorithmes d'apprentissage pour F.Poulard et N.Hernandez.

3.2.4 Ce qui nous intéresse dans l'article

Nous avons repris l'approche de (Hernandez et Poulard, 2008) pour la détection du discours repris à savoir la techniques d'apprentissage supervisé. Nous avons également retenu comme candidats au discours repris, uniquement les segments textuels entre guillemets.

3.3 L'article de R. Mitkov

3.3.1 Synthèse de l'article

L'article que nous allons synthétisé est "Robust pronoun resolution with limited knowledge " de Rusian Mitkov.
L'objectif de cet article est la résolution de l'anaphore d'un pronom donné, c'est à dire trouver le groupe nominal auquel il se réfère.
Les étapes de l'approche de Mitkov sont :

1. **Obtention des candidats** : Les candidats sont des groupes nominaux, on commence par récupérer tous les groupes nominaux dans les deux phrases précédents le pronom.
2. **Filtre, le genre et le nombre** : On élimine de la liste de candidats tous ceux ne correspondant pas en genre ou en nombre au pronom.
3. **On attribut des notes aux candidats** sur différents critère (indicateurs), et on propose comme résultat (antécédent) le candidat dont la note globale (ie la somme des résultats) est la plus élevée.
4. **En cas d'égalité** : on détermine l'antécédent selon la note d'un unique indicateur (l'indicateur le plus décisif) sur les candidats restants. Si l'on a toujours pas un unique résultat, on sélectionne selon le second indicateur par ordre d'importance, et ainsi de suite jusqu'à obtenir un seul candidat qui sera le groupe nominal de référence.

Ce qui est à retenir de cette approche c'est que le choix des indicateurs est crutial, ainsi que le poids de chacun des indicateurs par rapport aux autres.

3.3.2 Ce qui nous intéresse dans l'article

Nous pensons reprendre la méthode d'obtention des candidats et d'attribution des notes aux candidats. Le fait de déterminer le genre et le nombre des candidats nous semble difficilement faisable.

3.4 Le mémoire de F. Poulard

3.4.1 Contenu

Lors de son stage de Master 2 recherche en 2007, Fabien POULARD a travaillé sur le sujet suivant : "Le repérage automatique de citations dans les documents journalistiques".
Dans son rapport, il commence par définir la notion de citation puis il présente le corpus qu'il a constitué et présente ce qu'il a observé dans ce même corpus.
Ensuite, Fabien présente la mise en place des chaînes de traitement qui permettent l'identification des citations et des expressions locuteurs. Cette partie est particulièrement intéressante pour nous. Fabien explique qu'il choisit des techniques d'apprentissage supervisé puisqu'il est difficile de définir des régles de fonctionnement étant donné la complexité des langues. Après avoir présenté la chaîne de traitement, il introduit les composants d'extraction de candidat. Fabien repère les discours repris entre guillemets. Par la suite, il décrit les choix des attributs et des composants pour la caractérisation des segments entre guillemets (cadres mimétiques).
Voici les principaux attributs cités :

- **taillecadremim** : taille du cadre mimétique candidat en nombre de mots bruts.
- **distmimphrast-gauche** : distance entre le bord gauche du cadre phrastique et le bord gauche du cadre mimétique.
- **distmimphrast-droit** : distance entre le bord droit du cadre phrastique et le bord droit du cadre mimétique.
- **distplusprochecadre** : distance du cadre mimétique le plus proche dans le contexte phrastique.
- **enoncverb-within** : présence du verbe d'énonciation au sein du cadre mimétique candidat.
- **enoncverb-out** : présence du verbe d'énonciation en dehors du cadre mimétique candidat.

Fabien présente ensuite les expérimentations et évaluations qu'il a fait des chaînes d'identification. Il présente les algorithmes d'apprentissage étudiés et explique que son choix se portera sur l'algorithme J48 du fait de son efficacité.

Chapitre 4

Observation d'un corpus de textes

4.1 Objectifs de l'observation

Nous avons observé une partie du corpus de textes dans le but de trouver des règles pour la résolution d'anaphore pronominale. On cherche à dégager des caractéristiques des anaphores pronominales redondantes dans les articles observés.
Nous avons observé dix articles différents par personnes et comme nous étions trois à travailler sur cette partie on arrive à 30 articles observés. Nous part dans l'idée de définir un cadre pour l'anaphore pronominale avant le discours repris, de trouver une distance moyenne entre l'entité nommé et le discours repris. Nous avons décider de compter cette distance en nombre de phrase, c'est la donné qui nous paraissait la plus stable et la plus cohérente. On aurait pu mesurer en nombre de mot mais cette donnée semble varier beaucoup en fonction du style de l'article.
On espère également trouvé les entités nommée plutôt en début de phrase et en tant que sujet. On a compté la distance entre l'entité nommée et le début de la phrase et repéré les types de mots des entités nommées.
Tout comme Rusian Mitkov, nous avons commencé par observer si la première entité nommé trouvée correspondant en genre et en nombre au locuteur était la bonne.

4.2 Résultats

Nous avons commencé par observer la distance entre le discours repris et l'entité nommée référence dans le but de trouver un cadre de recherche.
Voici nos résultats :

L'entité nommé est	Nombre de cas	Pourcentage cumulé
Dans la même phrase que le discours repris	7	0
Dans la phrase précédent le discours repris	39	42
Deux phrases avant le discours repris	24	64
Trois phrases avant le discours repris	17	73
Quatre phrases avant le discours repris	9	88
Cinq phrases avant le discours repris	6	93
Six phrases avant le discours repris	0	93
Sept phrases avant le discours repris	4	97
Huit phrases avant le discours repris	1	98
Neuf phrases avant le discours repris	2	100
Total	109	

On observe que l'entité nommée référence est le plus souvent dans la phrase précédent le discours repris et 88% des entités nommées de référence se situe dans les quatre phrases qui précèdent le discours repris.

Dans 75% des cas l'entité nommé de référence est le groupe nominal correspondant en genre et en nombre le plus proche avant le discours repris. C'est le premier critère qu'utilise Mitkov pour la détection d'anaphore pronominale. Comme nous pouvons le constater, ce critère a une bonne pertinence. Malheureusement, c'est très difficile à mettre en place et nous n'aurons pas le temps de faire un outil déterminant le genre et le nombre d'une entité nommée.

Nous avons également observé la classe de mot correspondant à l'entité nommée référence. Dans 86% des cas l'entité nommée de référence est un sujet.

Pour rendre les informations concernant le type de mot de l'entité nommée référence plus pertinente nous avons également observé sa position dans la phrase. Dans la moitié des articles observés, l'entité nommée de référence commence la phrase. En moyenne l'entité nommée de référence se trouve a 25% de la phrase, en nombre de mot.

Nous avons ensuite observé la place de l'entité nommée de référence dans la phrase par rapport aux autres entités nommées. Dans 91% des cas, l'entité nommée de référence est la première de la phrase et dans 9% des cas, c'est la deuxième. On est content.

Nous avons également observé la place de l'expression locuteur par rapport discours repris. Dans 64% des cas, l'expression locuteur se situe avant le discours repris. Dans 33% des cas elle se situe après et dans 3% des cas l'expression locuteur se situe dans le discours repris.

4.3 À retenir

Les règles que nous allons pouvoir extraire de ces observations concernent :
- La place de l'entité nommée dans le texte c'est-à-dire à combien de phrases elle se situe par rapport au discours repris.
- La positon de l'entité nommée dans la phrase.
- La place de l'entité nommé dans la phrase par rapport aux autres.

Voici les deux idées qui ressortent de cette observation :

La première est de sélectionner les quatre phrases précédents le discour repris. Dans chaque phrase, s'il y a des entités nommées, on sélectionne la première. Si on a plusieurs entités nommées résultats, on prend celle qui est la plus proche du discours repris dans l'article.

La deuxième idée est de remonter dans le texte à partir de la phrase contenant le discours repris. Pour chaque phrase, s'il n'y a pas d'entité nommée, on passe à la phrase suivante. S'il y en a une, on la considère comme le résultat.

Chapitre 5

Approches

5.1 Précision du sujet

Le problème de la résolution d'anaphore dans le cadre de la détection de citation est très large, doit-ton résoudre toures les anaphores ? et ce pour tout les types de citations ?

Nous avons précisé le sujet en résolvant uniquement les anaphores **pronominales**, et en ne détectant que les discours repris **directs**

5.2 Sous-problèmes et nos choix

5.2.1 La détection de citation

Comme nous l'avons dit, notre approche d'une citation sera la même que celle présentée par Fabien Poulard et Nicolas Hernandez dans "Qui dit quoi ?" (citation = discours repris + expression locuteur + segment citationnel, sans notion de motif sur les composants).

La détection des expressions locuteurs : Les expressions locuteurs seront déjà annotées dans les textes à analyser.

La détection des discours repris : comme nous l'avons précisé au début de cette section, nous ne détecteront que les discours reprits directs c'est à dire encadrés par des guillemets. Pour cela nous avons utilisé un *algorithme d'apprentissage*.

L'algorithme va, sur la base de statistiques prédire si un candidat (un segment entre guillemets) est un discours repris ou non. Un algorithme d'apprentissage s'utilise en deux phases :

1. Pour obtenir les statistiques nous devons construire un modèle prédictif à partir d'un corpus déjà annoté appelé *corpus d'apprentissage* et selon un certains nombre de critères qui seront des attributs des candidats de l'apprentissage.
2. On caractérise nos candidats par un ensemble d'attributs (que l'on appelera instance), puis on envoi ces instances à l'algorithme d'apprentissage qui selon le modèle prédictif sera en mesure de nous dire si chacun des segments entre guillemets est un discours repris ou non.

   Pour paramétrer notre algorithme d'apprentissage nous avons utilisé les critères de caractérisation décrits dans le mémoire de F. Poulard

5.2.2 La résolution d'anaphores pronominales

Pour résoudre les anaphores pronominales contrairement à la détection des discours repris avec un algorithme d'apprentissage, nous avons opté pour une solution basée sur des règles. En effet l'observation du corpus nous a montré que 88% des entités nommées références d'un pronom sont rencontrées dans les 4 phrases précédant le discours repris (énoncé par le pronom).
De plus dans 3 cas sur 4 l'entité nommées la plus proche (la première trouvé en revenant en arrière dans le texte) et qui correspond en genre et en nombre à l'anaphore pronominale est bien l'entité nommée cherchée.
Ainsi avec quelques règle simples nous avons estimé pouvoir obtenir un résultat satisfaisant, voici donc les trois sous-problèmes auquels nous devons répondre :

La détection des anaphores à résoudre : Comme nous l'avons précisé dans la section sur la détection de citations, les expressions locuteurs seront déjà annotées dans les textes à analyser. Pour connaître les anaphores que nous devons effectivement résoudre, nous ferons une sélection sur ces expressions locuteurs, en gardant uniquement les pronoms.

La détection des candidats de la résolution : Les candidats de la résolution sont les entitées nommées, pour les détecté nous avons d'abords pensé à un système de reconnaissance spécialisé pour les entités nommées (Nemesis) mais par manque de temps nous avons dût simplifier cette étape, nous avons donc sélectionner tous les mots commencant par une majuscule (c'est à dire en utilisant des règles).

Le choix des règles pour la résolution d'anaphores : Pour le choix des règles nous nous somme inspiré de l'approche de Rusian Mitkov car nos observations de corpus nous ont montré de bons résultats en appliquant la première règle sur la corrélation entre le genre et le nombre d'une anaphore et du candidat. (dans : "Robust pronoun resolution with limited knowledge")
C'est donc ce que nous avons utilisé, nous avons considéré la première entité nommée rencontrée en remontant dans le texte (en dehors du discours repris) correspondant en genre et en nombre au pronom locuteur.

5.3 Implémentation

5.3.1 Choix techniques

UIMA

UIMA (Unstructured Information Management Architecture) est un framework de traitement de données non structurées d'abord lancé par IBM, qui est depuis en cours de normalisation par l'OASIS (Organization for the Advancement of Structured Information Standards). L'objectif de ce framework est de décrire les étapes de traitement d'un document non structuré (texte, image, vidéo, etc.), en vue d'en extraire de façon automatique des informations structurées. Ce framework peut être assimilé à un outil de text-mining car il a pour but d'automatiser l'extractiion de données structurés dans des documents peu ou non structurés. L'outil

de text-mining pourra donc généré de l'information sur le contenu du document qui n'était auparavant pas présente dans le document initial. Cette information sera alors rajouté afin d'enrichir le document original dans des finalités qui peut etre varié tel que :
– à classifier automatiquement des documents
– à avoir un aperçu du contenu d'un document sans le lire
– à dégager une information precise sur des corpus documentaires importants

UIMA est aussi un framework qui fournit l'environnement d'exécution dans lequel les développeurs peuvent intégrer leurs implémentations de composants UIMA pour construire et déployer des applications. UIMA fournit un kit de développement qui inclut une implémentation Java de l'architecture et du framework UIMA. Il existe un certain nombre d'élément de base qui premettent le travail sous le framework UIMA.

Les Analysis Engines (AE) sont des blocks de base de l'architecture UIMA utilisé pour l'-analyse de documents afin de détecter des attributs à l'intérieure de celui-ci. Ces informations produitent par l'AE sont appelé Analysis Results (AR). Les AR représentent des méta données qui on pu etre extraite suite à l'analyse du document par l'AE. Les AR sont des régions du document plus granulaire que le document initial dans sa globalité. Ainsi on peut définir un AR comme la séquence de caractère qui couvre un type de données que l'on cherche à extraire. Les AE sont des objets agrégés et gérés par le framework UIMA. Les développeurs développent des algorithmes d'analyses et de détection des types de données recherchés dans des composants appelé Annotators qui sont inclus dans les AE afin de les inclures dans le framework.

Les Type System sont des Types de données de base d'UIMA pouvant être étendus par les développeurs pour aboutir à un schéma plus riche de types, appelé Type System (TS). Un TS definie les types de données pouvant etre decouvert par les AE qui utilise ce TS. Un exemple de TS pourrait etre le type Personne. Un TS peut également avoir des caractéristiques propre. Toujours avec notre exemple du TS Personne les caractéristiques seraient l'age, le sexe ou bien la taille. Il n'y a pas de limite dans les TS qui peuvent etre definient, aussi ils pourront etre organisé un taxonomie où un type Employe sera un sous type de Personne.

Les Common Analysis Structure (CAS) sont des structures de données objet permettant de représenter et de partager les resultats entre les différentes partie de l'architecture UIMA. Les CAS contiennent le document qui est analysé ainsi que les instances objet des TS rencontrés dans le document. UIMA supporte l'analyse simultanée de plusieurs vues du même document dans le but d'effectuer une analyse multimodale, comme par exemple une vue pour analyser un TS et l'autre vue pour analyser un second TS du même document. Dans UIMA, une vue s'appelle Subject of analysis (Sofa). Une CAS contient dans ce cas un ou plusieurs Sofa avec les objets représentant les résultats d'analyse pour chaque Sofa.

En plus des composants de base, d'autres composants sont définis dans UIMA pour supporter tout le cycle de développement des applications. Les principaux sont :
– Collection Reader : se connecte à une source d'informations et itère sur la collection des documents qui la compose pour initialiser les objets CAS avant l'analyse.
– CAS Consumer : intervient à la fin du processus pour traiter les résultats d'analyse effectués par les différents AE et les rendre exploitables par d'autres applications, comme l'indexa-

tion des résultats pour un engin de recherche, le stockage dans une base de données, etc.

Il faut également savoir que deux parties sont essentielles pour mettre en place tout composant dans UIMA :
– La partie déclarative : qui décrit l'identité, la structure et le comportement du composant à l'aide de XML.
– La partie implémentation du composant à l'aide d'un langage de programmation comme Java

La figure suivante (figure 5.1 page 18) montre une vue d'ensemble du fonctionnement des applications qui utilisent UIMA.



FIG. 5.1 – Architecture Haut Niveau des composants UIMA

Weka

Weka est un logiciel open Source Java développé par les chercheurs de l'université de Waikato en Nouvelle Zélande. Il fourni un ensemble d'algorithme d'apprentissage pour le data-mining. Les algotithmes peuvent soit etre directement appliqué à un ensemble de données ou bien utilisé au sein d'un code source Java. Weka contient des outils de classification, clustering et visualisation de données. Weka fourni un ensemble d'algorithme de classification qui grace à un apprentissage supervisé permettra la génération d'arbres de décissions.

5.3.2 Notre travail

Afin de répondre au sujet nous devions implémenter des composants s'intégrants au framework UIMA afin de détecter d'un coté les discours repris et de l'autre permettre la résolution des anaphores. Pour réaliser cela nous avons séparer le travail suivant deux axes d'implémentations qui sont la détection des discours repris et la résolution des anaphores

Détection des discours repris

Dans la partie suivante nous utiliserons les techniques et approches développé par fabien Poulard dans son mémoire de Master 2 recherche (voir chapitre état de l'art).

Dans un premier temps nous avons implémenté le composant UIMA permettant la détection des candidats au discours repris. Ce composant est en réalité un enchainement de plusieur module UIMA formant une chaine de traitement. Cette chaine de traitement (figure 5.2 page 20) est composé d'un collection reader du corpus annoté, d'un tokenizer, d'un annoteur de verbes et de pronoms, d'un detecteur de candidats au discours repris, d'un comparateur des annotations, d'un générateur de fichier `arff` et d'un CAS consumer xmi.

Notre implémentation c'est essentiellement axé sur le developpement des modules de detection des candidats au discours repris, de comparaison des annotations et de génération de

fichier `arff`. En effet, les autres composants que nous avons utilisé en amont et aval de ces modules etaient déja implémenté et nous n'avont fait que les réutiliser.

Afin d'implémenter le module de détection des candidats au discours repris nous avons tout d'abord élaboré un algorithme de détection des zones entre guillemet ainsi que leurs caractérisation (algorithme 5.2 page 22). Cet algorithme a ensuite été retranscrit en JAVA et intégré dans un Analysis Engine UIMA. Pour fonctionner, ce module UIMA necessite de récupèrer les annotations généré par les modules en amont, a savoir le tokenizer et l'annotateur de verbes et pronoms afin de faire fonctionner son algorithme.

Listing 5.1 – Algorithme de détection des candidats au discours repris

```
pileDiscours = Pile<DiscoursRepris>
liste = Liste<DiscoursRepris>

pour chaque phrase faire

        nbMotQuote = 0;
        nbMotPhrase = 0;

        autreCadre = faux
        verbEnoncOut = faux
        pronomFirstOut = faux
        pronomThirdOut = faux

        pour chaque mot de la phrase faire
                nbMotPhrase++;
                nbMotQuote++;
                si mot est un pronom d'enonciation alors
                        si pileDiscours est vide alors
                                verbEnoncOut = vrai
                        sinon
                                pileDiscours.peek.verbEnoncIn = vrai
                        finsi
                finsi

                si mot est un pronom premiere personne alors
                        si pileDiscours est vide alors
                                pronomFirstOut = vrai
                        sinon
                                pileDiscours.peek.pronomFirstIn = vrai
                        finsi
                finsi

                si mot est un pronom troisieme personne alors
                        si pileDiscours est vide alors
                                pronomThirdOut = vrai
                        sinon
                                pileDiscours.peek.pronomThirdIn = vrai
                        finsi
                finsi
```

```
                si mot est un guillemet alors
                        si pileDiscours est vide alors
                                nbMotQuote=0
                                discoursRepris = new DiscoursRepris
                                discoursRepris.setBegin(mot.getBegin
                                    +1);
                                discoursRepris.setDistMimPhrastGauche(
                                    nbMotPhrase);
                                pileDiscours.push(discoursRepris);
                        sinon
                                discoursRepris = pile.pop;
                                discoursRepris.setTailleCadreMim(
                                    nbMotQuote);
                                discoursRepris.setEnd(word.getEnd-1);
                                discoursRepris.setDistMimPhrastDroit(
                                    nbMotPhrase);
                                liste.ajouter(discour);
                        finsi
                finsi
        finpour

        pour chaque DiscoursRepris de la liste faire
                discoursRepris.DistMimPhrastDroit(nbMotPhrase-discours
                    .getDistMimPhrastDroit());
                discoursRepris.AutresCadreMim(autreCadre);
                discoursRepris.EnoncVerbOut(verbEnoncOut);
                discoursRepris.PronomFirstSecondOut(pronomFirstOut);
                discoursRepris.PronomThirdOut(pronomThirdOut);
                discoursRepris.Classe(DISCOUR_REPRIS);
        finpour

        liste.clear
finpour
```

Nous avons ensuite implémenté le module de comparaison des annotations détectées par le module précédent avec les annotations du corpus. Ce module permet de comparer les annotations générées par le collection reader du corpus et celles générées par le module de détection des candidats au discours repris. La comparaison permet de déduire si oui ou non les candidats au discours repris détectés correspondent à un segment annoté comme discours repris dans le corpus. Si un candidat au discours repris détecté est inclut dans un segment annoté comme discours repris dans le corpus alors celui-ci est caractérisé comme Discours Repris.

Enfin nous avons implémenté le module d ration du fichier arff à partir des annotations de candidats au discours repris détecté. En e module necessite pour fonctionner l'execution préalable du module de détection des ca s au discours repris.

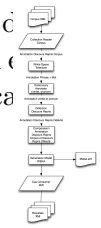


FIG. 5.2 – Chaine d'apprentissage des discours repris

Dans un second temps nous avons choisi d'implémenter une chaine de traitement se rapporchant énormement de celle précédement réalisée (figure 5.3 page 21). Cette chaine devait permettre de detecter comme la précédente les zones de texte entre guillemet. Cependant, cette chaine ne devait plus s'effectuer sur un corpus annoté mais sur un corpus de texte brut (non annoté). Afin de réaliser cette chaine de traitement nous avons quelque peut modifier le précédente. Celle-ci contient des modules commun à la précédente comme le tokenizer, l'annotateur de verbes et de pronoms, le detecteur de candidats au discours repris, le générateur de fichier `arff` et le CAS consumer xmi. Nous avons donc remplacé le collection reader du corpus par un collection reader de texte brut et supprimer le module de comparaison des annotations. Nous avons également du effectuer une petite modification dans le code du générateur de fichier `arff` afin que celui-ci prenne en compte si la chaine s'applique ou non sur le corpus annoté. Cette modification permet en effet de modifier le fichier de sortie du générateur (modele.arff ou test.arff).



FIG. 5.3 – Chaine de détection des discours repris

Résolution des anaphores

Il faut savoir que dans l'idéal, les modules de détection des discours repris et de résolution des anaphores sont étroitement liés. Un fois détectés, les discours repris devaient ensuite permettre d'en chercher les expressions locuteurs associées. Cette transition n'ayant pas pu être implémentée, le module de résolution des anaphores fonctionne indépendamment de la détection des discours repris.
Le procédé expérimental ne suit pas exactement les observations du corpus, les contraintes de temps nous ont astreint à expérimenter une méthode plus légère.

Procédé expérimental

Ici nous nous concentrons uniquement sur les anaphores pronominales. La résolution des anaphores est réalisée à partir du corpus d'articles de presse. Ce corpus présente l'avantage d'avoir les expressions locuteurs déjà annotées, ce qui nous facilite grandement la tâche dans l'annotation des anaphores.

La première étape est donc l'annotation des anaphores que nous allons tenter de résoudre au mieux. Pour se faire, nous ne gardons, parmi les expressions locuteurs, que les pronoms personnels puisque seule les anaphores pronominales nous intéressent.

La deuxième étape consiste à annoter les entités nommées. L'idée de départ était de les détecter grâce à l'outil Némésis, un système de reconnaissance des entités nommées pour le français. Le temps imparti nous étant insuffisant pour prendre en main correctement ce système, nous avons dû trouver une approche rapide à mettre en place pour les détecter, c'est

à dire une reconnaissance des mots débutants par une majuscule.

Enfin il s'agit d'expérimenter la méthode de résolution des anaphores à proprement dite. L'approche que nous proposons est de définir comme entité nommée source la dernière entité nommée avant l'anaphore (ou la première en partant de l'anaphore et en remontant le document).

Déploiement de la méthode proposée

Cinq éléments interviennent dans la chaine de résolution des anaphores : Le Collection Reader du corpus, Le White Space Tokenizer, le Dictionary Annotator, l'anaphore Analysis Engine, qui correspond au composant que nous avons crée puis en fin de chaine un CAS Consumer(figure 5.4 page 23).

Ce dont le composant AnaphoreAE a besoin en entré, c'est des expressions locuteurs ainsi que des pronoms annotés. Le dictionary anotator, qui a besoin du white space tokenizer sert à annoter les pronoms fournis par un dictionnaire.

AnaphoreAE annote les anaphores en faisant l'intersection entre les expressions locuteurs et les pronoms annotés par le dictionary.

La détection des entités nommée est réalisée par une expression régulière qui reconnait les mots commençant par une majuscule excepté les mots de début de phrase qui représentait une part très importante du bruit que nous pouvions avoir.

Listing 5.2 – Algorithme de détection des entités nommées et de résolution des anaphores

```
pile : Pile
pour chaque expression locteur dans le document faire
        pour chaque pronom dans le document faire
                si dans(pronom,expressionLocuteur) alors
                        annoter comme anaphore
                finsi
        finpour
finpour

pour chaque mot du document faire
        si anaphore(mot) alors
                si nonVide(pile) alors
                        attribuer sommet(pile) comme
                           entiteNommee source
                finsi
        sinon
                si commenceParMajuscule(mot) etnonFiniParPoint
                   (precedent(mot)) alors
                        mot est entiteNommee
                        ajoute( entiteNommee,pile)
                finsi
        finsi
finpour
```



FIG. 5.4 – Chaine de résolution des anaphores

Chapitre 6

Évaluation

6.1 Détection du discours repris

Afin d'évaluer notre implémentation nous avons procédé à deux types d'expérimentation.

Dans un premier temps nous avons comparé les segments annoté par notre composant avec ceux annoté manuellement dans le corpus de texte journalistique. Cette méthode a été effectué grâce à l'outil intégré à UIMA de visualisation d'annotation. Nous avons donc procédé au traitement d'un lot de texte du corpus par notre composant de détection des candidats au discours repris puis nous avons choisi aléatoirement certain d'entre eux afin de consulter les annotations apporté à celui-ci et de les comparer avec les annotations déja présente dans le corpus. La figure suivante (figure 6.1 page 24) présente l'interface graphique Annotation Viewer UIMA qui nous a permis d'effectuer la comparaison des annotations.



FIG. 6.1 – Evaluation de l'implémentation avec l'Annotation Viewer UIMA

Dans la figure précédente on peut voir que l'interface se divise en trois parties disctinctes. Une première où le texte analysé est affiché avec la coloration des zones annotées. Une seconde où on peut choisir quelle type d'annotation nous souhaitons visualiser. Enfin une troisième où lorsque l'on clique sur une annotation dans la zone de texte affiche les informations de cette annotation.
Nous avons donc utilisé cette outil afin de comparer les deux types d'annotations. Nous avons donc regardé si les annotations générées par notre composant se superposaient avec les annotations du corpus. En cliquant sur nos annotations détecté nous avons regardé si elles elles correspondaient à un segment annoté du corpus. Dans la figure précédente on peut voir dans la zone à droite (affichant les informations) que les annotations générées par notre composant (en bleu) se superposent bien sur les annotations du corpus (en violet) en comparant les attributs de début et de fin des annotations. Dans l'exemple on voit donc que le segment entre guillemet :

Nous venons demander un procès équitable (...) inquiets quant à l'issue du procès

est à la fois une annotation détecté par notre composant et aussi une annotation du corpus.

Dans un second temps nous avons choisi d'évaluer notre solution grâce à des algorithme d'apprentissage. En effet, le nombre d'attributs par annotations ainsi que le nombre de valeurs pour chacun d'entre eux étant assez important nous avons choisi d'utilisé cette alternative pour évaluer notre implémentation. En effet, nous aurions pu utiliser une méthode à base de règle pré-définies pour classer nos instances d'annotations mais nous ne savions pas réellement quels attributs étaient significatifs pour le classement.
Nous avons donc procédé au traitement d'un lot de texte du corpus par notre composant de détection des discours repris qui a généré un fichier `arff` interprétable par weka. Nous avons ensuite réalisé l'interprétation de ce fichier par weka en utilisant l'implémentation de l'algorithme C4.5 de weka, J48. Cette algorithme de classification de type arbre de décision nous a donc permis de générer un arbre de décisions suivant les différents attributs significatifs de nos instances d'annotations (figure 6.2 page 25).



FIG. 6.2 – Arbre de décision généré par Weka

Une fois cette arbre de décision généré nous avons pu testé celui ci suivant deux techniques. La première en utilisant la cross-validation, la seconde en utilisant un ensemble de test (figure 6.3 page 25).
La cross-validation permet d'évaluer le taux d'erreur d'un apprentissage en échantillonant le modèle d'apprentissage en n sous-échantillons puis en effectuant le test de ceux-ci dans l'arbre de décisions. Le resultats retourné est alors la moyenne des resultats de l'ensemble des sous-échantillons. Pour effectuer la validation croisée nous avons choisi de découpé notre ensemble en 10 sous-échantillons.
La seconde technique utilisé pour tester l'arbre de décision a été réalisé en procedant à la seconde chaine de traitement sur un ensemble de texte brut (figure 5.3 page 21). Nous avons ensuite utilisé le fichier `arff` généré par cette chaine pour l'évaluation de notre implémentation.



FIG. 6.3 – Evaluation de l'implémentation avec Weka

L'ensemble des résultats de ces expérimentations seront discutés dans le chapitre suivant, analyse des résultats.

6.2 Résolution des anaphores pronominales

Nous avons étudiés les résultats sur dix articles du corpus contenants tous au moins une anaphore détectée. L'objectif était dans un premier temps d'analyser l'efficacité de la méthode de détection des entités nommées puis la résolution des anaphores.

La méthode de détection des entités nommée n'était pas un objectif en soit. Cependant, leur détection est nécessaire pour le fonctionnement de la résolution d'anaphore. Si nous analysons les résultats de cette détection, c'est donc principalement dans le but de d'étudier la résolution des anaphores.

Voici les critères d'observation des résultats :

- Nombre entité nommées détectées
  - Nombre de fausse
- Nombre d'anaphores
  - Nombre de bien résolues
  - Nombre de mal résolues
    - Pour cause de mauvaise détection des entités nommées
    - Du à la technique de résolution

Nous nous intéressons particulièrement au bruit de la détection des entités nommées, qui avec les premières observations s'est avéré conséquent.

Chapitre 7

Analyse des résultats

7.1 Détection du discours repris

Comme nous l'avons vu dans le chapitre d'évaluations de la solution nous avons utilisé deux types d'évaluations pour notre composant de détection des discours repris.

La première méthode d'évaluation, à savoir la comparaison des annotations du corpus avec les annotations générées par le composant nous à fourni des résultats plutôt concluant dans le sens ou sur l'ensemble des document observés les annotations détectées par le composant correspondaient dans une grande majorité à des segment où des discours repris avait été repéré manuellement.

La seconde méthode d'évaluation, à savoir l'utilisation d'un algorithme d'apprentissage à lui aussi été concluant puisqu'il nous a fourni un taux de classification des instances d'annotations des discours repris pour la validaion croisée de :

Correctly Classified Instances	399	83.6478 %
Incorrectly Classified Instances	78	16.3522 %
Total Number of Instances	477	

Tab. 7.1 – Tableau des instances classées par l'algorithme d'apprentissage

D'après ces résultats nous pouvons donc dire que 83.6478 % des annotations discours repris détecté correspondent bien à un discours repris alors que 16.3522 % d'entre elles ne correspondent pas. Le fait ou non qu'une instance soit bien classée depend de son cheminement dans l'arbre de dérivation. En effet, si les attributs de l'instance la mêne vers une classification mais que celle ci ne correspond pas à la classe de l'instance, elle est alors considéré comme mal classé. Au contraire si l'arbre mène l'instance vers une classification qui correspond à sa classe, elle est alors considéré comme bien classé. La cross-validation nous également apportée des informations sur les valeurs de précision et de rappel de chaque classe. La précision et le rappel sont des notions de la statistique qui permettent de quantifier la pertinence dans les recherches documentaires. En effet, la **précision** calcul le nombre de documents pertinents trouvés par rapport au nombre total de documents sélectionnés alors que le **rappel** lui calcul

le nombre de documents pertinents trouvés par rapport au nombre total de documents pertinents.
On trouve ainsi les valeurs suivante pour la précision et le rappel pour chaque classe d'instance détectée.

Classe	Precision	Recall	F-Measure
Discours Repris	0.856	0.605	0.709
Non Discours Repris	0.831	0.95	0.886
Weighted Avg.	0.839	0.836	0.828

TAB. 7.2 – Tableau des taux de precision et rappel de chaque classe

7.2 Résolution des anaphores pronominales

Bruit parmi les entités nommées : 19,44 %

Anaphores bien résolues : 16,67%

Mal résolue à cause du bruit parmi les entités nommées : 20,83%

Mauvaise résolution due à la méthode : 62,5%

Le bruit parmi les entités nommées est relativement important. Dans la méthode d'annotation des entités nommées, tout les mots (ou token) suivant un token terminant par un point ne sont pas annotés. Il s'agit là d'éliminer tous les mots de début de phrase qui commencent systématiquement par une majuscule. Cependant, sur certains documents, l'indentation fait que le token précédent n'est pas un mot terminant par un point mais un espace vide.

« Oublié de l'Etat», Baloua ne se laisse plus faire. Mais il ajoute(...)

Le « Mais » est ici annoté comme entité nommée à tort en raison de l'indentation. L'anaphore il sera dans ce cas associé à Mais. On observe également qu'une proportion très importante des anaphores est mal résolue à cause de la méthode elle-même : on associe la dernière entité nommée rencontrée. Exemple de problème de résolution :

(...) a dit David Martinon lors de son point de presse hebdomadaire. "Le président Chavez voudra sans doute informer le président Sarkozy de l'état de ses efforts et ils essaieront de voir comment faire avancer ce processus pour que cesse le calvaire enduré par les otages étrangers des Farc", a-t-il ajouté.

Ici le « il » fait référence à David Martinon. Cet exemple montre que les entités nommées au sein de la citation ne sont probablement pas de bons candidats.
Dans l'exemple suivant, on voit que l'identification du genre et du nombre aurait pu aider à résoudre certaines anaphores :

Benazir Bhutto proposait à son rival des années quatre-vingt-dix, l'ancien chef du gouvernement Nawaz Sharif, en exil, de former avec elle une alliance contre le président Pervez Musharraf. À propos des autres leaders de l'opposition, elle a déclaré à des journalistes au téléphone(...)

Notre composant a ici annoté « elle » comme anaphore faisait référence à Musharraf.

Chapitre 8

Conclusion

8.1 Sur notre travail

Même si durant toute la durée du module nous avons eût l'impression que le temps nous manquait pour avoir une solution satisfaisante, le problème de la détection de citation assez bien été résolu au regard des résolutats obtenus (83% d'efficacité). Ce à quoi nous pouvions nous attendre car nous connaissions l'efficacité de cette approche avec le rapport de master de Fabien Poulard.

Le résultat de la résolution d'anaphores est quant à lui insufisant (mois de 17% d'efficacité) et cela principalement à cause d'une approche très simpliste et incomplète (nous n'avons pas eût le temps de mettre en place une règle vérifiant le genre et nombre des entités nommées par rapport au pronom).

8.2 Nos observations du travail de recherche

Approche scientifique Le travail de recherche sur un s'effectue selon les phases suivantes :

(a) Le travail bibliographique, la recherche de publications pour appréhender l'état de l'art concernant une problématique et pour avoir un aperçu des sous-problèmes quelle comporte. Cette phase est très longue et très importante pour en pas aboutir sur un échec. Dans notre cas le travail bibiographique et le recadrage du sujet ont occupés au moins 80% du temps.

(b) Après avoir cerné le sujet, soit on utilise une solution existante en l'adaptant, soit on propose une solution nouvelle. La première phase doit bien sûr avoir été suffisament poussée pour ne par réinventer une solution déjà existante.

(c) En recherche informatique et plus particulièrement en TAL, les productions des chercheurs sont souvent des méthodes, des approches d'un problème donné, la phase suivante consiste donc a implémenter un programme métant en oeuvre leur méthode afin de prouver son efficacité ou sont innefficacité à défaut de preuve formelle (c'est particulièrement vrai en TAL où la plupart des méthodes ne sont pas exactes à 100%).

Dans notre groupe de recherche le travail a été partagé efficacement selon les préférences de chacun et la phase d'implémentation a été facilité par l'utilisation d'un dépôt SVN auqueltous les membres ont participés.

(d) Ensuite vient la phase d'évaluation de la méthode ou du programme selon un protocol bien défini.

(e) Enfin l'analyse des résultats et discution sur la pertinence de la solution

On notera que l'important pour faire de la recherche est :

- De savoir travailler en équipe car c'est presque toujours selon ce mode que fonctionne les chercheurs
- Avoir un esprit de synthèse pour savoir expliquer son point de vue aux autres membres de l'équipe, pour assimiler de grandes quantités de connaissances nouvelles et pouvoir rédiger des publications. Avoir un esprit synthétique permet aussi de fixer les limites d'un problème à résoudre et gagner beaucoup de temps.

Table des figures

5.1 Architecture Haut Niveau des composants UIMA 18
5.2 Chaine d'apprentissage des discours repris 20
5.3 Chaine de détection des discours repris 21
5.4 Chaine de résolution des anaphores 23

6.1 Evaluation de l'implémentation avec l'Annotation Viewer UIMA 24
6.2 Arbre de décision généré par Weka 25
6.3 Evaluation de l'implémentation avec Weka 25

Liste des tableaux

7.1 Tableau des instances classées par l'algorithme d'apprentissage 27
7.2 Tableau des taux de precision et rappel de chaque classe 28